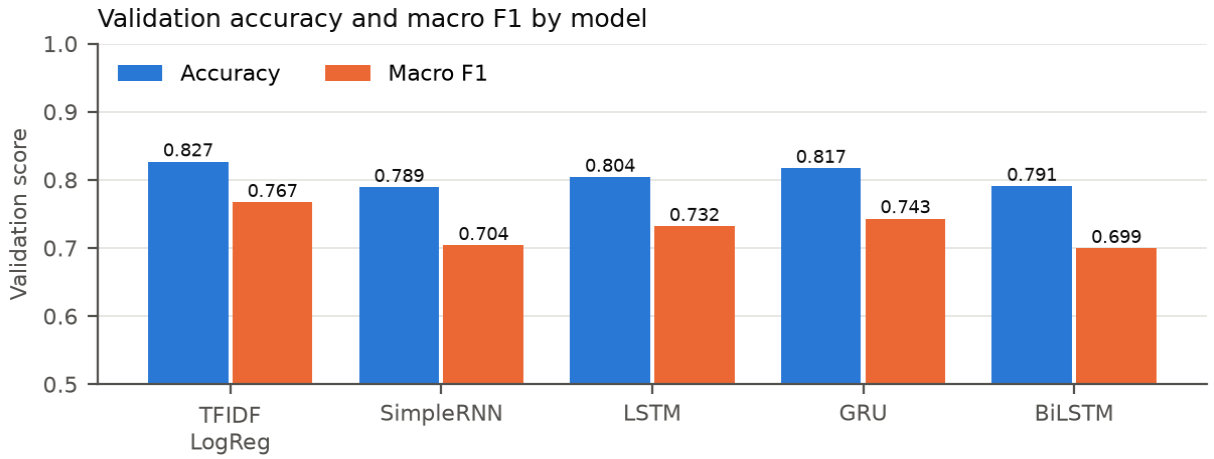
Financial Tweet Sentiment: RNN baselines vs BERT

Dataset: zeroshot/twitter-financial-news-sentiment (9,543 train / 2,388 validation tweets; Bearish, Bullish, Neutral). All scores are on the official validation split, which was never used for training or early stopping.

1. Results

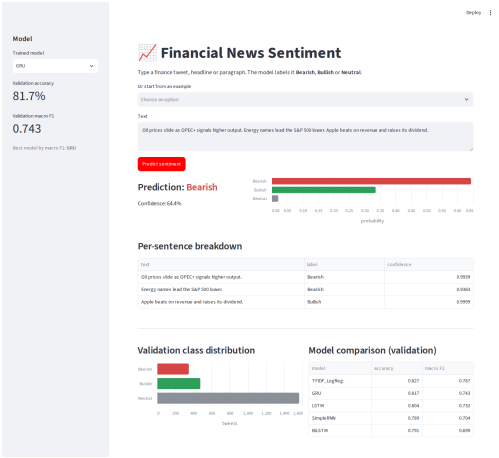
Model	Params	Accuracy	Macro F1	Bearish F1	Bullish F1	Neutral F1
TFIDF_LogReg	-	0.827	0.767	0.679	0.733	0.889
SimpleRNN	0.95M	0.789	0.704	0.586	0.651	0.874
LSTM	1.05M	0.804	0.732	0.600	0.715	0.881
GRU	1.02M	0.817	0.743	0.621	0.722	0.886
BiLSTM	1.18M	0.791	0.699	0.563	0.658	0.878
BERT (fine-tuned)	110M	pending	pending			



BERT status: the fine-tuning code (src/bert_finetune.py, notebook section 6) is complete and was verified end-to-end with a tiny offline model, but the pre-trained weights could not be downloaded in the build environment (no Hugging Face access). Run *python -m src.bert_finetune* on a GPU (Colab: about 10 minutes), then *python scripts/make_report.py*; this paragraph, the table and the charts update with the BERT numbers automatically.

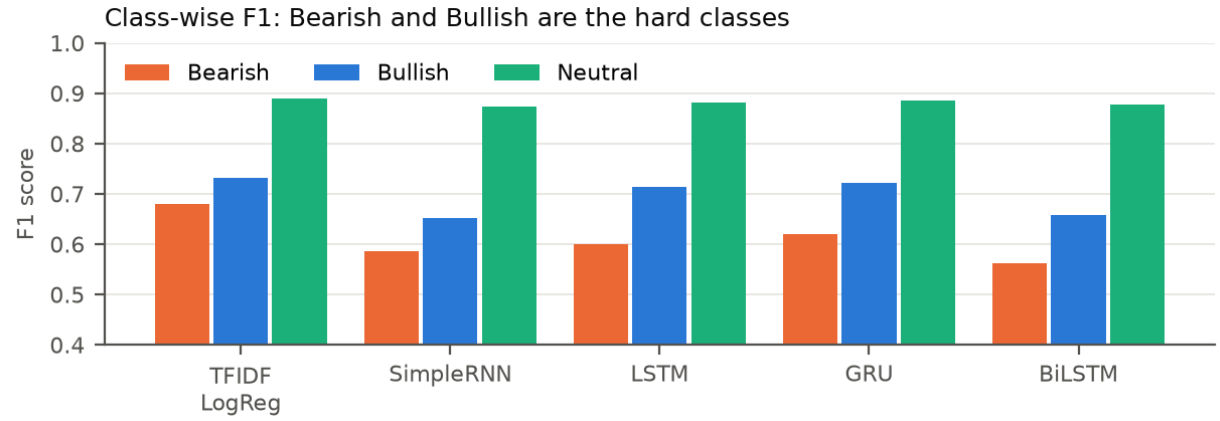
2. Which RNN variant works best and why

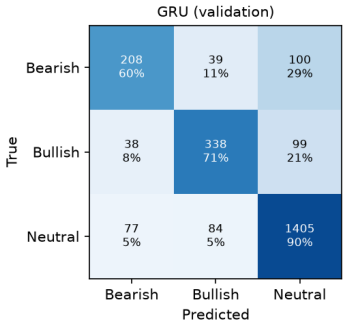
GRU is the best recurrent model (macro F1 0.743, accuracy 0.817). The gated cells (LSTM, GRU) beat the vanilla RNN because their gates keep a sentiment word seen early in the tweet from being overwritten by later neutral tokens such as tickers and source tags; the plain RNN trains less stably and needed twice the epochs. GRU matches LSTM with fewer parameters, which suits a small dataset. The BiLSTM fit the dev slice well but generalised worse on validation: with about 8.6k training tweets the extra capacity mostly memorises. A TF-IDF + logistic regression reference scores 0.767 macro F1, ahead of every from-scratch RNN: on short headlines, sentiment is carried by a few keywords, and randomly initialised embeddings cannot learn rare keywords from so little data. That gap is precisely what pre-trained BERT is meant to close.

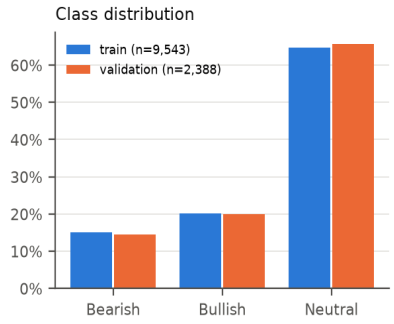


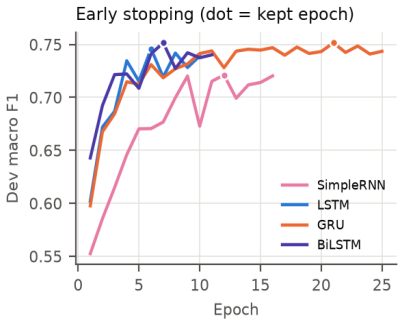
Streamlit dashboard (app/streamlit_app.py): choose a trained model, type a tweet, headline or paragraph, and get the label, a class-probability bar chart and a per-sentence breakdown, plus the validation class distribution and the model leaderboard. Here a three-sentence paragraph is scored by the GRU: two bearish energy sentences outweigh one bullish Apple sentence, so the paragraph is labelled Bearish (64%).

3. Where the models fail









For GRU, 29% of Bearish and 21% of Bullish tweets are predicted Neutral, while Bearish and Bullish are rarely confused with each other: the model mostly misses *weak* sentiment rather than flipping polarity. Typical causes in the most confident errors: (1) **implicit sentiment** with no polar keyword ("Transport for London has decided not to extend Uber's license"); (2) **domain knowledge**, where a word flips meaning ("Traders are dumping virus hedges" is bullish); (3) **quotes and hedged language** ("the rebound can be robust"); (4) **label noise** on borderline headlines. The RNNs are also over-confident (probabilities near 1.0 on mistakes) because dev loss rises after a few epochs even while dev F1 still improves.

- *Press release: cyber attack - update January 31, 2020 #Picanol #Stock #MarketScreener https://t.co/AXwQmr* (true Bearish, predicted Neutral)
- *The Estée Lauder Companies Delivers Exceptional Fiscal 2020 Second Quarter Results* (true Bullish, predicted Neutral)
- *"There is every reason to believe that the economic rebound, when it comes, can be robust," Fed Chair Jer* (true Bullish, predicted Neutral)
- *Stock Traders Are Dumping Virus Hedges After 'Peak Fear' Passes* (true Bullish, predicted Bearish)

4. Design choices and the Streamlit app

Leakage control: 10% of TRAIN (stratified) is a dev slice for early stopping; the vocabulary is built from the remaining training tweets only. Imbalance (65% Neutral) is handled with inverse-frequency class weights, which is why macro F1, not accuracy, drives model selection. Each RNN uses a 128-d embedding, one 128-unit recurrent layer, final-state + max-pool readout, dropout 0.4, Adam (lr 2e-3, halved on plateau), gradient clipping and patience-4 early stopping. The Streamlit dashboard (app/streamlit_app.py) lets the user pick any trained model, type a tweet or paragraph, and see the predicted label, a class-probability bar chart, a per-sentence breakdown, the validation class distribution and the model leaderboard; src/predict.py offers the same paragraph-level inference from the command line.